Siligent Dental AI Assistant Handoff

1. Product Summary

Siligent Dental AI Assistant is a local-first dental front-office AI application. It helps administrative users generate and review dental communications using local Ollama models, structured templates, uploaded context, and controlled runtime fields.

Current production path:

- Frontend: React + TypeScript + Vite, served through Nginx in Docker.
- Backend: FastAPI.
- Database: PostgreSQL with pgvector.
- Model runtime: Ollama running on the host machine.
- Deployment: Docker Compose for `frontend`, `api`, and `db`; Ollama is not containerized.
- Primary user roles: `admin` and `staff`.

Current product workflows:

- Insurance denial letter generation.
- Insurance verification.
- Email exchange reply drafting.
- Template library management.
- Model routing/settings.
- System health and audit review.

2. Repository Map

Important locations:

- `frontend/` - React application.
- `frontend/src/pages/` - Main UI pages.
- `frontend/src/components/` - Reusable UI components.
- `frontend/src/lib/` - Frontend API client, auth, formatting, runtime field helpers, and task state.
- `api/main.py` - FastAPI app, route registration, auth dependencies, request handlers.
- `api/schemas.py` - API request and response schemas.
- `services/` - Backend business logic, storage, model calls, prompt handling, generation, guardrails.
- `prompts/` - Version-controlled prompt files used by the app.
- `data/payer_references/` - Local payer reference files used for insurance verification.
- `docs/` - Product-specific supporting documents and field mapping specs.
- `docker-compose.yml` - Runtime composition for database, backend, and frontend.
- `start.sh` - Starts the Dockerized app stack after checking host Ollama.
- `stop.sh` - Stops the Dockerized app stack and attempts to offload Ollama models.
- `.env.example` - Environment variable template.
- `requirements.txt` - Python dependencies.
- `frontend/package.json` - Frontend dependencies and build scripts.
- `tests/` - Backend test suite.

3. Runtime Architecture

The browser loads the React app from `http://localhost:3000`. The React app calls the FastAPI backend at `http://localhost:8000/api`. The backend stores structured application data in PostgreSQL and stores uploaded file content under `data/uploads` through the mounted `./data:/app/data` volume.

Ollama runs directly on the host at `http://localhost:11434`. Inside Docker, the API reaches Ollama through `http://host.docker.internal:11434`.

Runtime services:

- `db`: PostgreSQL 16 with pgvector, exposed on host port `5434`.
- `api`: FastAPI backend, exposed on host port `8000`.
- `frontend`: React static app served by Nginx, exposed on host port `3000`.
- `ollama`: Host process, expected on host port `11434`.

4. Environment Configuration

Start from:

```bash
cp .env.example .env
```

Important variables:

- `OLLAMA_URL`: Host Ollama URL for non-container execution. Compose overrides this to `http://host.docker.internal:11434`.
- `MODEL_NAME`: Default local model, currently `qwen3.5:4b`.
- `OLLAMA_GENERATE_TIMEOUT_SEC`: Generation timeout, currently `180`.
- `OLLAMA_NUM_PREDICT`: Max model output tokens, currently `1024`.
- `OLLAMA_THINK`: Whether to use model thinking mode, currently `false`.
- `OLLAMA_KEEP_ALIVE`: Ollama model keep-alive, currently `0` to unload after use.
- `AUTH_ENABLED`: Auth toggle, currently `true`.
- `AUTH_SESSION_HOURS`: Session duration, currently `12`.
- `ALLOW_SELF_REGISTER`: Self-registration toggle, currently `false`.
- `API_KEY`: Optional API key gate. Blank means API key check is disabled.
- `CORS_ORIGINS`: Frontend origin, currently `http://localhost:3000`.
- `DATABASE_URL`: Postgres connection string.
- `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_PORT`: Compose database settings.

For deployment beyond local development, change default database credentials before starting the stack.

5. Starting and Stopping

Prerequisites:

- Docker Desktop or Docker Engine is running.
- Ollama is installed on the host.
- The selected model is pulled locally.

Pull the default model if needed:

```bash
ollama pull qwen3.5:4b
```

Start the app:

```bash
./start.sh
```

Stop the app:

```bash
./stop.sh
```

Manual Compose commands:

```bash
docker compose up -d --build
docker compose ps
docker compose logs -f api
docker compose down
```

Access points:

- App: `http://localhost:3000`
- API health/docs: `http://localhost:8000/docs`
- Database: `localhost:5434`
- Ollama: `http://localhost:11434`

6. Authentication and Roles

Authentication is enabled by default.

Bootstrap behavior:

- If no users exist, the first registered user becomes the bootstrap admin.
- After bootstrap, additional account creation is restricted.
- Admin users can create new users from the model/settings administration UI.

Roles:

- `admin`: Can manage shared templates, template types, model preferences, users, field dictionary entries, and audit/system areas.
- `staff`: Can use generation workflows and personal templates, but cannot manage shared system-level configuration.

There are no hardcoded production credentials in the application. Credentials are created through the bootstrap/register flow.

7. Current User Workflows

Denial Letter Generation

Route/page:

- Frontend page: denial letter workflow inside the composer experience.
- Backend endpoint: `POST /api/denial-letters/generate`.

User flow:

1. Select or open the denial letter workflow.
2. Enter denial-specific fields such as patient, payer, procedure, claim, denial code, denial reason, and appeal basis.
3. Generate the draft.
4. Review and edit the generated letter.
5. Copy, download, or print/save as PDF.

Current behavior:

- Letter type is treated as denial letter for this workflow.

- Denial code descriptions are resolved from backend constants.
- Runtime fields are normalized before generation.
- The backend attempts to prevent unsupported factual invention for high-risk va
lues.

Insurance Verification

Route/page:

- Frontend page: insurance verification workflow.
- Backend endpoint: `POST /api/insurance-verification/generate`.

User flow:

1. Select payer.
2. Enter patient/member details.
3. Enter requested condition and requested procedure when relevant.
4. Generate verification.
5. Review coverage summary and verdict.

Current behavior:

- Payer reference content comes from `data/payer_references/*.txt`.
- The app produces a structured verification response.
- It includes a coverage verdict for the requested condition/procedure based on
the available local payer reference text.
- If the reference text does not support coverage, the app should avoid presenti
ng unsupported coverage as fact.

Email Exchange Drafting

Route/page:

- Frontend page: email exchange/thread workflow.
- Backend endpoint: `POST /api/email-thread/generate`.

User flow:

1. Paste email thread content or provide relevant uploaded context.
2. Provide recipient, sender, topic, next step, and other runtime details.
3. Generate a reply draft.
4. Review and edit before sending externally.

Current behavior:

- The app does not connect directly to a mailbox.
- The workflow is manual input/upload based.
- The generated draft should be written from the Siligent provider/front-office
perspective.
- Guardrails reject unresolved missing markers such as `Not provided` in the fin
al reply.

Template Library

Route/page:

- Frontend page: template library.
- Backend endpoints: `GET /api/templates`, `POST /api/templates`, `DELETE /api/t
emplates/{index}`.

User flow:

1. Browse templates by type/tags.
2. Open a template in the workspace.
3. Edit wording and variables.
4. Save as personal or shared depending on role.

Current behavior:

- Shared templates are admin-managed.
- Staff can create and manage personal variants.
- The app enforces one shared canonical template per purpose type.
- Templates use natural-language variable labels in the UI, while backend rendering resolves canonical placeholders.

Model Settings

Route/page:

- Frontend page: model/settings administration.
- Backend endpoints: `GET /api/models`, `GET /api/model-preferences`, `PUT /api/model-preferences`.

Current behavior:

- Admin users can configure model preferences.
- Preferences can be global or use-case-specific.
- Models are read from the local Ollama installation.
- The configured model must already exist locally in Ollama.

System Health and Audit

Route/page:

- Frontend page: system health.
- Backend endpoints: `GET /api/health`, `GET /api/audit-events`.

Current behavior:

- Health checks validate API and model configuration state.
- Audit events are available to admins.
- Audit logs should avoid patient data and focus on actions, users, and operational metadata.

8. Prompt and Generation System

Prompt files live under `prompts/`.

Prompt routing is centralized in:

- `services/prompt_registry.py`

Generation-related services:

- `services/generation.py` - denial letters, email drafts, insurance verification, template drafts.
- `services/prompt_engine.py` - prompt loading/rendering helpers.
- `services/template_runtime.py` - runtime field normalization and template rendering.
- `services/document_pipeline.py` - multi-document extraction and smart composer support.
- `services/email_thread.py` - thread analysis and reply generation.
- `services/email_guardrails.py` - email draft validation.
- `services/autonomy_policy.py` - factual grounding controls.

- `services/ollama_client.py` - Ollama health/model/generation API client.

Operational rule:

- Prompt text should remain in `prompts/**/*.txt`.
- Runtime code should select prompts through the prompt registry.
- Do not scatter prompt strings directly across UI or route handlers.

9. Data and Persistence

Primary persistence:

- PostgreSQL through `DATABASE_URL`.

Postgres-backed stores include:

- Users and sessions.
- Templates.
- Template types.
- Field dictionary.
- Model preferences.
- Upload metadata.
- Audit events.

Filesystem data:

- `data/payer_references/` - payer coverage reference text files.
- `data/uploads/` - uploaded source files and extracted document content.
- `prompts/` - read-only prompt templates mounted into the API container.

File-store fallback classes still exist in `services/`, but the current Docker r
untime uses Postgres.

10. API Surface

Primary endpoints:

- `GET /`
- `GET /api/auth/bootstrap`
- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/auth/me`
- `GET /api/health`
- `GET /api/models`
- `GET /api/model-preferences`
- `PUT /api/model-preferences`
- `GET /api/template-types`
- `POST /api/template-types`
- `GET /api/field-dictionary`
- `PUT /api/field-dictionary/{field_key}`
- `DELETE /api/field-dictionary/{field_key}`
- `GET /api/denial-codes`
- `GET /api/payers`
- `POST /api/denial-letters/generate`
- `POST /api/emails/generate`
- `POST /api/email-thread/generate`
- `POST /api/insurance-verification/generate`
- `POST /api/dentrix/resolve-template-fields`
- `POST /api/document-pipeline/generate`
- `POST /api/template-drafts/generate`
- `GET /api/templates`

- `POST /api/templates`
- `DELETE /api/templates/{index}`
- `GET /api/audit-events`

API errors are returned as structured JSON with an error code and human-readable message.

11. Frontend Structure

Main files:

- `frontend/src/App.tsx` - route composition.
- `frontend/src/components/AppShell.tsx` - top-level layout/navigation.
- `frontend/src/lib/api.ts` - API client.
- `frontend/src/lib/auth.tsx` - auth state.
- `frontend/src/lib/generationTasks.tsx` - generation task state persistence.
- `frontend/src/lib/runtimeFields.ts` - runtime field helpers.
- `frontend/src/lib/templateVariables.ts` - template variable helpers.
- `frontend/src/styles.css` - global visual system.

Main pages:

- `InsuranceVerificationPage.tsx`
- `SmartComposerPage.tsx`
- `TemplateLibraryPage.tsx`
- `ModelSettingsPage.tsx`
- `SystemHealthPage.tsx`
- `LoginPage.tsx`

12. Verification Checklist

Run backend tests:

```bash
python3 -m unittest discover -s tests -p 'test_*.py' -v
```

Run frontend build:

```bash
cd frontend
npm run build
```

Check Docker stack:

```bash
docker compose ps
```

Check API:

```bash
curl -sS http://localhost:8000/
```

Check Ollama:

```bash
ollama ps
ollama list
```

Manual workflow smoke test:

1. Open `http://localhost:3000`.
2. Bootstrap/login as admin.
3. Confirm `/api/health` works through the System Health page.
4. Generate one denial letter.
5. Generate one insurance verification with payer reference data.
6. Generate one email exchange reply from pasted thread content.
7. Edit a generated draft.
8. Save a personal template.
9. As admin, save/update a shared template.
10. Stop the app with `./stop.sh` and confirm models are offloaded with `ollama ps`.

13. Known Operational Constraints

- Ollama must be running on the host before generation works.
- The selected model must already be pulled locally.
- Vision-capable model behavior depends on the local model installed in Ollama.
- Insurance verification depends on local payer reference files; unsupported payer data cannot be inferred reliably.
- Generated content must be reviewed by a human user before being sent, printed, or filed.
- The app is intended for local/private deployment, not public internet exposure without additional infrastructure hardening.

14. Maintenance Notes

Adding a payer:

1. Add a text file under `data/payer_references/`.
2. Use lowercase filename formatting where spaces become underscores, for example `delta_dental.txt`.
3. Restart or refresh the app if the payer list does not immediately appear.

Adding or changing a prompt:

1. Edit the relevant file under `prompts/`.
2. Confirm the prompt is registered in `services/prompt_registry.py`.
3. Run backend tests.
4. Run a manual generation test for the affected workflow.

Adding a template type:

1. Admin creates the new type through the UI, or it is added through the template type store path.
2. Add or update the shared canonical template for that type.
3. Confirm required variables are understandable to staff users.

Changing model routing:

1. Pull the model locally with Ollama.
2. Confirm it appears in the Model Settings page.
3. Set global or per-use-case preference.
4. Run a generation smoke test.

15. Handoff Acceptance Criteria

A new owner should be able to accept the project when they can:

- Start the app with `./start.sh`.

- Log in or bootstrap the first admin account.
- Create a staff account.
- Generate and edit a denial letter.
- Generate and interpret insurance verification output.
- Generate and edit an email exchange reply.
- Save and update templates according to role permissions.
- Change model settings to an installed local Ollama model.
- Run backend tests and frontend build successfully.
- Stop the stack cleanly with `./stop.sh`.